

Agentic AI

Code with
- Arohi

* Why Traditional AI's fail?

- ↳ Spam mail filter → might fail to detect new types.
have to manually update
- ↳ Customer Support chatbot → out of context questions.
It cannot adapt to new situations / learn from interactions without human input.

* Agentic AI -

It is a type of AI that can make decisions and take actions on its own.

- handle complex & unpredictable situation.
- adapt and learn over time.
- make autonomous decisions

examples: autonomous vehicles, netflix, spotify, etc.

* Generative AI -

all about creating new content based on its training data. eg - chatgpt, DALL-E, GANs

* How Gen AI and Agentic AI used together?

↓
creates new

↓
decision-making
adaptability

learning over time
(autonomy)

↗ Tesla cars

Example 1: Advertising and Marketing

Agentic AI - analyses customer data to decide which ads to show.

Gen AI - creates personalized ads

Agentic AI chooses the target audience. Gen AI designs the ad content

* What are AI agents & LLM agents?

⇒ agent can perceive, reason and act from the env.

1. AI agents - eg: recommendation system, robot vacuum

1. understand user's behaviour
2. decides what to recommend
3. taking action

collaborative
filtering
content-based
filtering

* NLP X , uses AI models, structured data

2. LLM agents - designed to work with NLP and perform task based on text input.

1. Understand user's query

ex: query - "Tell me the weather of Delhi"

intent: get weather data

entity: delhi

2. decides what needs to be done to answer the user's query.

example: use Weather API to fetch

3. taking action

example : gets data in raw format, HM will be used to get user friendly reply.

* Types of Agents -

agent loop : perception $\rightarrow$ decision $\rightarrow$ action

1. Simple Reflex agent
2. Model based reflex agent
3. Goal-based agent
4. Utility-based agent
5. Learning agent

1. Simple Reflex agent : basic type of agent

- only reacts
- no memory
- no planning

example :

Automatic door
Motion detect lights.

2. Model based reflex agent :

- has memory $\rightarrow$ model of the world
- current input isn't sufficient

3. Goal-based agent :

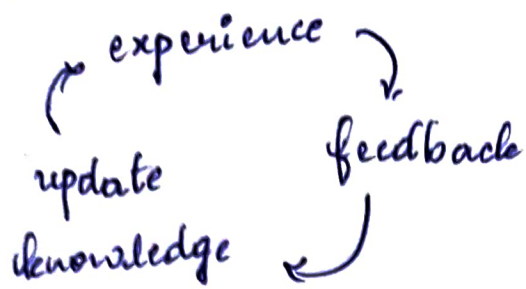
- ✓ reacts, memory
- It only plans

example : GPS tracking
flight booking assistant

4. Utility-based agents :

- $\hookrightarrow$ goal-based agent + smart optimisation
- $\hookrightarrow$ Scores to choose options towards high probability reward.

5. Learning agents :
improve over time.



example: losses $\rightarrow$ trial, better $\rightarrow$ next game.

19/08/26

Reinforcement learning $\rightarrow$

getting rewards for good actions, getting penalties for bad actions.

example: training a doggy

1. observe the environment
2. agent takes action
3. the environment gives feedback
4. learn from feedback and adjusts its future decisions to get more rewards.

Environment :

1. update state
2. reward agent

State: snapshot of the env at specific time.

state $\xrightarrow{\text{action}}$ next state

Action: Possible move taken by agent.

Reward: number --- +ve $\rightarrow$ good, -ve $\rightarrow$ bad/penalty

Policy: rulebook that agent follows

Types of Policy

Deterministic Policy

↳ always choose the same action for a given state.

Stochastic Policy

↳ choose an action based on probabilities.

* Value function $V(s)$ -

$V(s)$ means - "How good it is for the agent to be in a particular state".

How does the Agent learn these values?

↳ by playing, through its states.

High $V(s)$ = agent wants to go there again

Low $V(s)$ = agent avoids it.

How to Calculate $V(s)$?

↳ Bellman Expectation equation:

$$V(s) = E_{\pi}[R + \gamma V(s')]$$

$V(s)$ - value of current state

R - immediate reward

γ - discount factor (how much we care abt future rewards)

$V(s')$ - value of next state

π - policy

E - Expectation (avg over possible outcomes)

$$R = +5, \quad \gamma = 0.9, \quad \gamma(b') = 10$$

$$v(s) = 5 + 0.9 \times 10$$
$$= 14$$

Initially it doesn't know these values:

we use learning algorithms,

1. Monte Carlo
2. Temporal difference (TD) learning
3. Q-learning

*Q-learning - is a model-free reinforcement learning algorithm

model-free : agent doesn't know $R, \gamma, v(s')$.

→ Q-Value ? $Q(s, a)$
 $\swarrow$ state
 $\searrow$ action

→ Q-table ? actions

	$\downarrow a_1$	$\downarrow a_2$	$\downarrow a_3$
$s_1 \rightarrow$	$Q(s, a)$	"	"
$s_2 \rightarrow$	"	"	"
$s_3 \rightarrow$	"	"	"

14	2	7
0.01	-5	-10
0.4	-15	+15

example: Snake - food

State	Left	Right	Forward
Near food	+10	-20	+2
Near wall	-100	-50	0
Open space	+1	+2	+3

→ How Q-learning actually work?

$$Q(s, a) \leftarrow \underbrace{Q(s, a)}_{\text{current/old state}} + \underbrace{\alpha}_{\text{learning rate}} \left[\underbrace{r}_{\text{reward}} + \underbrace{\gamma \cdot \max_{a'} Q(s', a')}_{\text{discount factor} \cdot \text{next state}} - Q(s, a) \right]$$

$$= r + \gamma \cdot \max_{a'} Q(s', a')$$

$$= 10 + 0.9 \times 8 = 17.2$$

compute target

$$= [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

$$= 17.2 - 5 = 12.2$$

compute error

update the Q-value : $Q(s, a) = 5 + 0.1(12.2)$

$$= 5 + 1.22 = 6.22$$

$$5 < 6.22 = \text{improving}$$

20/08/26

Agentic AI Frameworks

* LangGraph -

→ open-source framework for building LLM-based applications.

→ It helps to build agents that can think, decide, use tools/APIs, take actions.

- # Node - a task / tool / action
 - # Edge - connection between nodes
 - # State - information of nodes
- } components of langGraph

State management -- remember details from previous activities.